

Mini-AlphaZero sur Puissance 4

Recherche Monte-Carlo guidée par un réseau policy + value, entraîné par self-play

Amine Rouibi

Mohamed Zouad

Thomas Sinapi

Université Paris Dauphine – PSL

Monte Carlo Search

1^{er} août 2026

Résumé

Nous implémentons, dans le périmètre du cours *Monte Carlo Search*, la chaîne complète qui va du playout aléatoire à un AlphaZero miniature, sur le Puissance 4 : moteur avec hachage de Zobrist et table de transposition, Flat Monte Carlo, UCB à la racine, UCT récursif, AMAF / RAVE / GRAVE, alpha-bêta avec table de transposition, puis réseau résiduel à deux têtes, PUCT, self-play parallélisé et boucle d'entraînement. Trois des cinq critères fixés à l'avance sont atteints, dont le critère décisif : à zéro simulation, la seule tête policy bat l'agent aléatoire 93,8 % du temps. PUCT à 100 simulations fait jeu égal avec UCT à 800 playouts, ce qui chiffre l'apport du réseau à un facteur 8 sur le budget de recherche ; et le réseau redécouvre seul, par pur self-play et sans aucune connaissance experte, que la colonne centrale est le meilleur coup d'ouverture – un résultat connu du jeu résolu – avec une probabilité de 0,914. Les deux critères manqués font l'objet d'un diagnostic complet : un premier entraînement a échoué, et son analyse occupe la section 8.

Table des matières

1	Introduction	2
1.1	Pourquoi Monte-Carlo	2
1.2	Ce que fait ce projet	3
1.3	Critères de réussite et résultat global	3
2	Le jeu et sa modélisation	3
3	Recherche Monte-Carlo sans réseau	6
3.1	Playout et Flat Monte Carlo	6
3.2	UCB à la racine	6
3.3	UCT	6
3.4	Résultats : l'échelle du cours (E1)	7
3.5	Où la différence se joue vraiment	7
3.6	Constante d'exploration (E2)	7
3.7	AMAF, RAVE et GRAVE (E8)	8
4	De AlphaGo à AlphaZero	9
4.1	Les quatre réseaux d'AlphaGo	9
4.2	AlphaGo Zero : ce qui est retiré	9
4.3	PUCT	9
4.4	Notre transposition au Puissance 4	10

5	Implémentation	10
5.1	Encodage	10
5.2	Réseau	10
5.3	Conventions de signe — la source de bug n°1	11
5.4	Self-play et parallélisation racine	11
5.5	Pourquoi pas de bitboards	11
5.6	Coût par coup (E9)	12
6	Protocole expérimental	12
7	Résultats	13
7.1	Les deux runs	13
7.2	Courbes d'apprentissage (E3)	13
7.3	Force au fil des itérations (E4)	14
7.4	Force en fonction du budget (E5) — la figure centrale	14
7.5	Accord avec le jeu parfait (E6)	15
7.6	Le réseau redécouvre la colonne centrale (E7)	16
8	Discussion et limites	16
8.1	Le run 1 : anatomie d'un échec	16
8.2	Le run 2 : la correction et ce qu'elle a donné	17
8.3	Ce qui n'est toujours pas résolu	18
8.4	Variance des mesures	18
8.5	Coût de calcul	18
9	Perspectives : les techniques du cours que nous n'avons pas implémentées	18
10	Conclusion	19
A	Reproduire les résultats	20
B	Organisation du code	20

1 Introduction

1.1 Pourquoi Monte-Carlo

L'alpha-bêta suppose une bonne fonction d'évaluation. Aux échecs on en a une : la valeur matérielle des pièces explique déjà l'essentiel de la position. Au Go on n'en a pas, et c'est précisément le point de départ du cours : trente ans de programmes de Go fondés sur la connaissance experte plafonnent au niveau débutant, jusqu'à ce que **Gobble** [4] propose de remplacer l'évaluation par de l'**échantillonnage** — jouer la partie au hasard jusqu'au bout, un grand nombre de fois, et préférer le coup dont les parties aléatoires finissent le mieux.

L'idée est contre-intuitive : les parties aléatoires sont d'une qualité catastrophique. Mais leur *moyenne* porte de l'information, et surtout elle ne demande aucune connaissance du jeu. Quand on n'a pas de bonne fonction d'évaluation, **échantillonner bat énumérer**.

Le cours suit ensuite l'enchaînement qui mène de là à AlphaGo Zero :

1. **Flat Monte Carlo** — répartir les playouts également entre les coups ;
2. **UCB à la racine** — traiter le choix du coup comme un bandit manchot, et allouer les playouts là où ils sont informatifs ;
3. **UCT** [5] — appliquer UCB *récurivement*, à chaque nœud de l'arbre ; c'est la rupture qui fait passer le Go de 10 kyu à 2 dan ;
4. **AMAF / RAVE / GRAVE** — partager l'information entre coups pour accélérer la convergence quand les statistiques sont rares ;
5. **AlphaGo → AlphaGo Zero → AlphaZero** — remplacer le playout aléatoire par un **réseau de neurones à deux têtes**, et remplacer le biais UCT par le *prior* de la tête policy : c'est PUCT.

1.2 Ce que fait ce projet

Nous implémentons la transposition directe des slides « **Alpha Zero Project** » et « **Projet Python** » du cours, qui décrivent le pipeline en cinq étapes pour Breakthrough 5×5 :

- 1) définir le réseau — 2) implémenter PUCT avec le réseau — 3) faire jouer l'algorithme contre lui-même — 4) enregistrer les distributions Monte-Carlo et le résultat des parties — 5) entraîner le réseau, puis itérer.

Nous remplaçons Breakthrough 5×5 par le Puissance 4, pour des raisons de coût de calcul : le facteur de branchement est de 7 au lieu d'environ 20, et les parties sont plus courtes (21 coups en moyenne en jeu aléatoire, contre plusieurs dizaines). Le pipeline, lui, est rigoureusement inchangé. Tout tourne sur CPU, sur une machine à 8 cœurs.

1.3 Critères de réussite et résultat global

Les critères ont été fixés **avant** de lancer les expériences. Voici le résultat final, mesuré sur le réseau final avec 100 simulations :

Table 1 — Critères de réussite, fixés avant les expériences et mesurés sur le réseau final à 100 simulations.

Critère	Seuil	Mesuré	IC Wilson 95 %	parties	Atteint
C1 — PUCT vs Aléatoire	98,0 %	100,0 %	[93,6 %, 100,0 %]	56	oui
C2 — PUCT vs UCT (100)	70,0 %	61,6 %	[48,5 %, 73,2 %]	56	non
C3 — PUCT vs alpha-bêta d.4	60,0 %	53,1 %	[43,3 %, 62,6 %]	98	non
C4 — Réseau seul vs Aléatoire	85,0 %	93,8 %	[84,2 %, 97,7 %]	56	oui
C5 — pertes décroissantes	qualitatif	2,29 → 1,68 sur 14 itérations			oui

Trois critères sur cinq sont atteints. C2 et C3 ne le sont pas, mais leurs intervalles de confiance à 95 % contiennent le seuil visé ($70\% \in [48,5; 73,2]$ et $60\% \in [43,3; 62,6]$) : avec le nombre de parties dont nous disposons, nous ne pouvons pas conclure que le seuil est manqué, seulement que nous ne pouvons pas démontrer qu'il est atteint. La §8 leur est consacrée, et s'appuie sur un **premier run raté** que nous rapportons intégralement.

C4 est le critère décisif : C1 à C3 pourraient passer avec un réseau nul mais une recherche correcte ; **seul C4 prouve que le réseau a appris quelque chose**, puisqu'il mesure la politique seule, à zéro simulation. Il est atteint largement.

2 Le jeu et sa modélisation

Le jeu : Grille de 6 lignes et 7 colonnes. Chaque joueur pose à tour de rôle un pion dans une colonne, où il tombe sur la première case libre. Le premier qui aligne quatre pions (horizontalement,

verticalement ou en diagonale) gagne ; si les 42 cases sont remplies sans alignement, la partie est nulle.

Deux propriétés comptent pour nous :

- **Le facteur de branchement est 7 au maximum**, et décroît quand les colonnes se remplissent. C'est très inférieur au Go (≈ 250) ou à Breakthrough 5×5 . La recherche couvre donc beaucoup de terrain avec peu de simulations.
- **Le jeu est résolu** [2, 3] : avec un jeu parfait, **le premier joueur gagne**, en jouant la colonne centrale au premier coup. Cette propriété a deux conséquences directes sur ce rapport : d'une part elle impose le protocole d'ouvertures équilibrées de la §6, sans quoi tous les chiffres seraient faussés ; d'autre part elle nous donne une vérité terrain gratuite pour l'expérience E7.

Mesure de contrôle du moteur : sur 5 000 parties aléatoires, JAUNE gagne 2 825 fois, ROUGE 2 161 fois, et 14 parties sont nulles. L'avantage du premier joueur est donc déjà visible en jeu purement aléatoire.

Représentation et interface : Nous suivons le style du cours : une classe Board avec legalMoves, play, terminal, score et playout (section Breakthrough).

Listing 1 – L'état de jeu.

```

1 ROWS, COLS = 6, 7
2 EMPTY, YELLOW, RED = 0, 1, 2      # YELLOW joue en premier
3
4 class Board:
5     cells: list[int]      # 42 cases, index = row*7 + col, ligne 0 = BAS
6     heights: list[int]    # 7 hauteurs de colonne
7     turn: int
8     moves: int
9     h: int                # hash de Zobrist
10    last: tuple | None    # (ligne, colonne) du dernier coup
11    winner: int | None

```

Convention de score. score() renvoie une valeur dans [0, 1] du point de vue de JAUNE : 1.0 si JAUNE gagne, 0.0 si ROUGE gagne, 0.5 pour une nulle. C'est exactement la convention du cours (1.0 if White wins, 0.0 else, 0.5 draw), et nous l'avons conservée telle quelle pour pouvoir transcrire sans adaptation les codes UCT, RAVE, GRAVE et PUCT du cours, y compris l'idiome if board.turn == RED: Q = 1 - Q, qui revient dans les quatre algorithmes.

Détection de victoire incrémentale. Après chaque coup, on ne teste que les alignements passant par la dernière case remplie. Nous précalculons, pour chacune des 42 cases, la liste des alignements de 4 qui la contiennent (69 alignements au total sur la grille) ; le test est alors une boucle sur environ 6,6 alignements. C'est $O(1)$ par coup.

Ordre des coups. legalMoves() renvoie les colonnes dans l'ordre [3, 2, 4, 1, 5, 0, 6] — le centre d'abord, comme le recommande le cours pour l'ordonnancement alpha-bêta. Tous les agents départagent donc les ex æquo de la même manière, ce qui rend les comparaisons plus propres.

Hachage de Zobrist [21] et table de transposition : Le cours pose la question : **combien de nombres aléatoires faut-il ?** Un par couple (couleur, case), plus un pour le trait :

$$2 \times 42 + 1 = 85 \text{ entiers de 64 bits.}$$

C'est beaucoup moins que les 789 nombres que le cours compte pour les échecs (6 pièces $\times$ 2 couleurs $\times$ 64 cases, plus le trait et les droits de roque) — parce qu'il n'y a ici qu'un seul type

de pièce et 42 cases.

`play()` met à jour le hash exactement comme le code du cours, en XOR-ant dans `self.h` les deux nombres `ZOBRIST[player-1][idx]` et `ZOBRIST_TURN`. Les nombres sont tirés à partir d'une **graine fixe**, ce qui est indispensable ici : le self-play est multi-processus et tous les workers doivent s'accorder sur les hashes.

Vérifications :

- le hash maintenu par XOR est identique au hash recalculé de zéro, sur les 63 464 coups de 3 000 parties ;
- **zéro collision sur 200 012 positions distinctes**, ce qui est attendu dans un espace de 2^{64} ;
- deux ordres de coups différents menant à la même position partagent bien la même entrée (`[0,1,2,3]` et `[2,3,0,1]`). Notons au passage qu'il faut **quatre** coups pour transposer au Puissance 4 : `[3,2]` et `[2,3]` donnent aux deux couleurs des cases opposées, ce ne sont pas les mêmes positions.

La table de transposition est un `dict[int, Entry]` indexé par `board.h`, réinitialisé à chaque coup joué à la racine — exactement le `Table = {}` du `BestMoveUCT` du cours. La disposition de l'entrée reprend la liste du cours :

Listing 2 – Disposition d'une entrée de la table de transposition.

```

1 class Entry:
2     n: float # t[0] playouts totaux
3     nplayouts: np.ndarray(7) # t[1] playouts par coup
4     nwins: np.ndarray(7) # t[2] victoires par coup
5     namaf: np.ndarray(84) # t[3] playouts AMAF (RAVE / GRAVE)
6     wamaf: np.ndarray(84) # t[4] victoires AMAF
7     prior: np.ndarray(7) # PUCT : tête policy
8     value: float # PUCT : tête value

```

Taux de hit mesuré (E9) : 77,5 % pour UCT, 82,6 % pour GRAVE, 78,4 % pour PUCT. Autrement dit, dans plus de trois descentes sur quatre, la position atteinte est déjà connue de la table. Le Puissance 4 est effectivement très riche en transpositions, puisque toute permutation de l'ordre des colonnes jouées mène à la même position. L'alpha-bêta n'affiche que 13,2 % parce qu'il explore un arbre beaucoup plus étroit, fortement élagué.

Codes de coups pour AMAF : Pour AMAF il faut un « code » identifiant un coup indépendamment du moment où il est joué. Le choix naïf serait **la colonne** : 7 codes. Nous ne l'avons pas retenu, et c'est délibéré.

Un playout de Puissance 4 dure une vingtaine de coups et joue donc **presque toutes les colonnes**. Avec 7 codes seulement, chaque code serait crédité dans presque chaque playout, les statistiques AMAF de tous les coups convergeraient vers la même valeur, et l'heuristique n'apporterait aucun signal.

Nous codons donc **la case remplie et le joueur** :

$$\text{code} = \text{joueur} \times 42 + \text{ligne} \times 7 + \text{colonne} \quad \text{soit 84 codes.}$$

C'est l'analogue direct du `Move.code` du cours pour Breakthrough (`5*(Dy*x1+y1)+direction`, dédoublé par couleur) : le code identifie *où* le coup atterrit, pas seulement *dans quelle direction* on le joue. Deux pions de la même couleur dans la même colonne mais à des hauteurs différentes sont des coups différents, ce qui est exactement la sémantique voulue.

3 Recherche Monte-Carlo sans réseau

Nous avons implémenté les quatre premiers barreaux de l'échelle du cours, dans l'ordre, en transcrivant les extraits de code des slides.

3.1 Playout et Flat Monte Carlo

Le playout est uniforme et joue jusqu'à une position terminale. `flat(board, n)` répartit les `n` playouts également entre les coups légaux et renvoie le coup de meilleure moyenne.

3.2 UCB à la racine

`UCB(board, n)` traite la racine comme un bandit à 7 bras et alloue chaque playout au coup maximisant

$$\bar{X}_i + c \sqrt{\frac{\ln i}{n_i}}$$

C'est le dilemme exploration / exploitation du cours : exploiter le bras qui semble le meilleur, mais explorer assez pour ne pas rester sur une illusion statistique.

3.3 UCT

`UCT(board)` applique la même règle **récurivement** à chaque nœud, avec la table de transposition. Nous reprenons le code du cours tel quel :

Listing 3 – Le cœur de UCT, transcrit du cours.

```

1 def UCT(board, table, c=0.4):
2     if board.terminal():
3         return board.score()
4     t = table.look(board)
5     if t is not None:
6         for m in board.legalMoves():
7             ni = t.nplayouts[m]
8             if ni == 0:
9                 val = INF                                # tout coup est essayé une fois
10            else:
11                Q = t.nwins[m] / ni
12                if board.turn == RED:
13                    Q = 1.0 - Q                            # convention du cours
14                val = Q + c * sqrt(log(t.n) / ni)
15            ...
16        board.play(best)
17        res = UCT(board, table, c)                        # récursion
18        t.n += 1; t.nplayouts[best] += 1; t.nwins[best] += res
19        return res
20    table.add(board)
21    return board.playout()                                # expansion + simulation

```

Deux écarts, purement notationnels : l'entrée est un petit objet plutôt qu'une liste (`t.n` pour `t[0]`), et les tableaux par coup sont indexés par **colonne** plutôt que par rang dans `legalMoves()` — au Puissance 4, le coup *est* la colonne, et les tableaux font déjà 7 cases.

Un détail qui a son importance pour lire les chiffres : **la première simulation ne fait que créer la racine** et jouer un playout, sans rien incrémenter. Après `n` simulations, la racine totalise donc `n - 1` visites.

3.4 Résultats : l'échelle du cours (E1)

Toutes à 200 playouts, 56 parties à ouvertures équilibrées par confrontation.

Table 2 – E1 — l'échelle du cours à 200 playouts, en confrontation deux à deux (56 parties à ouvertures équilibrées par ligne).

A	B	victoire de A	IC Wilson 95 %	V/N/D	Elo
Aléatoire	Flat MC (200)	5,4 %	[1,8 %, 14,6 %]	3/0/53	−499
Aléatoire	UCB racine (200)	0,0 %	[0,0 %, 6,4 %]	0/0/56	−3600
Aléatoire	UCT (200)	0,0 %	[0,0 %, 6,4 %]	0/0/56	−3600
Flat MC (200)	UCB racine (200)	58,0 %	[45,0 %, 70,0 %]	31/3/22	+56
Flat MC (200)	UCT (200)	25,9 %	[16,2 %, 38,6 %]	14/1/41	−183
UCB racine (200)	UCT (200)	32,1 %	[21,4 %, 45,2 %]	17/2/37	−130

La progression du cours est nette : n'importe quelle méthode Monte-Carlo écrase l'aléatoire, et **UCT domine largement les deux méthodes qui n'explorent qu'à la racine** (74,1 % contre Flat MC, 67,9 % contre UCB). En revanche Flat MC et UCB sont statistiquement indiscernables ici (58,0 %, IC [45,0 ; 70,0] qui contient 50 %). Ce n'est pas surprenant à 200 playouts et 7 coups : allouer intelligemment 28 playouts par coup ne change pas grand-chose ; ce qui change tout, c'est de descendre dans l'arbre.

3.5 Où la différence se joue vraiment

Une mesure complémentaire éclaire ce résultat. Sur 30 positions où le joueur au trait a un **gain immédiat**, et 30 positions où il doit **parer une menace immédiate** sous peine de perdre au coup suivant :

Table 3 – Tactique élémentaire : gains immédiats pris et pertes immédiates parées, sur 30 positions de chaque type, selon le budget.

algorithme	gains pris /30	parades /30		
		200	400	1000
Flat MC	30	23	26	28
UCB racine	30	26	26	27
UCT	30	29	29	30
GRAVE	30	29	30	30
RAVE	30	30	30	30
PUCT	30	30	30	30

Tous les algorithmes prennent un gain immédiat sans exception : il suffit d'un playout pour voir la partie se terminer sur une victoire. **Parer est beaucoup plus difficile pour les méthodes de racine**, et la raison est instructive : ne pas parer rapporte quand même environ 0,5 en playout aléatoire, parce qu'un adversaire aléatoire oublie très souvent de punir. Avec $200/7 = 28$ playouts par coup, cette différence se noie dans le bruit. Seule une recherche en arbre, qui concentre son budget là où c'est décisif, la résout — et UCT atteint 30/30 dès qu'on lui donne 1 000 playouts. C'est exactement la raison pour laquelle UCT bat Flat MC dans le tableau E1.

3.6 Constante d'exploration (E2)

Sur la grille {0,2 ; 0,4 ; 0,7 ; 1,0} contre l'alpha-bêta de profondeur 4, les scores sont respectivement 25,0 %, 18,8 %, 28,6 % et 13,4 %. **Les intervalles de confiance se recouvrent tous largement** sauf pour $c = 1,0$, nettement moins bon. Nous ne pouvons donc pas départager 0,2, 0,4 et 0,7 avec 56 parties ; on observe seulement que trop explorer nuit clairement. Nous

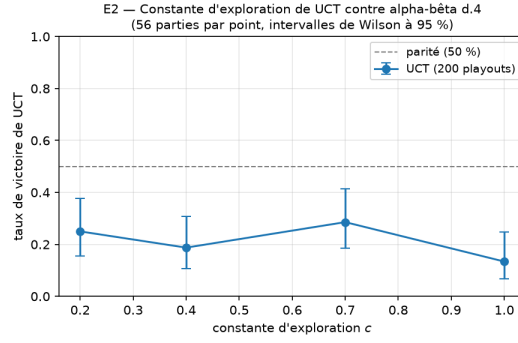


Figure 1 – E2 — Constante d'exploration de UCT contre l'alpha-bêta de profondeur 4, 56 parties par point, intervalles de Wilson à 95 %.

conservons $c = 0,4$, la valeur du cours, qui est bien calibrée pour des scores dans $[0, 1]$ — et nous notons honnêtement que nos mesures ne la valident pas contre 0,2 ou 0,7, elles n'ont simplement pas la puissance statistique pour le faire.

3.7 AMAF, RAVE et GRAVE (E8)

Nous avons transcrit `playoutAMAF`, `updateAMAF`, `RAVE` et `GRAVE(board, played, tref)` du cours. `updateAMAF` ne crédite que la **première occurrence** de chaque code dans le playout. Le poids de mélange est celui du cours [6] :

$$\beta = \frac{n_{\text{AMAF}}}{n_i + n_{\text{AMAF}} + b n_i n_{\text{AMAF}}}, \quad b = 10^{-5}$$

GRAVE [7] utilise les statistiques AMAF du **plus proche ancêtre ayant au moins ref playouts** (`ref` = 50 comme dans le cours). Avec `ref` = 0, chaque nœud est son propre référent et GRAVE redevient RAVE : c'est notre **vérification de cohérence**, et elle passe — sur 25 positions et 300 playouts, à graine fixée, GRAVE(`ref`=0) et RAVE produisent **exactement** la même distribution de visites.

Table 4 – E8 — UCT, RAVE et GRAVE (`ref` = 50) à 200 et 1000 playouts.

budget	confrontation	victoire de A	IC Wilson 95 %	V/N/D
200	GRAVE vs UCT	53,6 %	[40,7 %, 66,0 %]	29/2/25
200	RAVE vs UCT	49,1 %	[36,5 %, 61,8 %]	26/3/27
200	GRAVE vs RAVE	49,1 %	[36,5 %, 61,8 %]	27/1/28
1000	GRAVE vs UCT	35,7 %	[24,5 %, 48,8 %]	18/4/34
1000	RAVE vs UCT	47,3 %	[34,8 %, 60,1 %]	25/3/28
1000	GRAVE vs RAVE	47,3 %	[34,8 %, 60,1 %]	24/5/27

C'est un résultat négatif. À 200 playouts, GRAVE, RAVE et UCT sont indiscernables (tous les IC contiennent 50 %). À 1 000 playouts, **GRAVE devient significativement moins bon que UCT** (35,7 %, IC [24,5 ; 48,8] — la borne haute reste sous 50 %).

L'explication tient à l'hypothèse même d'AMAF : *la valeur d'un coup dépend peu du moment où il est joué*. Cette hypothèse est raisonnable au Go, où une pierre posée sur une intersection y reste et garde à peu près la même fonction. **Elle est beaucoup plus faible au Puissance 4**, pour une raison structurelle : une case n'est jouable qu'à une hauteur donnée, et seulement quand toutes les cases en dessous sont remplies. Jouer la case (3, 4) « plus tard dans le playout » n'est pas le même coup que la jouer maintenant — c'est un coup qui n'était même pas légal au moment où on l'évalue. Le crédit AMAF mélange donc des situations qui ne sont pas comparables, et introduit un biais que les vraies statistiques doivent ensuite corriger. Plus le budget augmente,

plus les vraies statistiques deviendraient fiables toutes seules, et plus le biais AMAF résiduel coûte cher : d'où la dégradation entre 200 et 1 000 playouts.

S'ajoute un coût pratique : GRAVE prend **27,2 ms par coup contre 8,1 ms pour UCT** à budget égal (E9), soit plus de trois fois plus, à cause de la mise à jour des tableaux de 84 codes à chaque playout. À temps égal plutôt qu'à playouts égaux, le verdict serait encore plus défavorable.

4 De AlphaGo à AlphaZero

4.1 Les quatre réseaux d'AlphaGo

Le cours décrit AlphaGo [9] comme la combinaison de quatre réseaux : une *policy* supervisée apprise sur des parties de joueurs forts, une *policy* rapide pour les playouts, une *policy* raffinée par apprentissage par renforcement, et un réseau de *value*. La recherche combine la valeur du réseau et le résultat d'un playout guidé.

4.2 AlphaGo Zero : ce qui est retiré

AlphaGo Zero [10] est surtout remarquable par ce qu'il **supprime** :

- plus de parties humaines : uniquement du self-play depuis l'initialisation aléatoire ;
- plus de connaissance experte en entrée : le plateau brut ;
- plus de playout du tout : la valeur de la feuille est donnée par la **tête value** ;
- un seul réseau à **deux têtes** (policy et value) partageant une tour résiduelle.

La recherche devient **PUCT** : le terme d'exploration d'UCT est remplacé par un terme proportionnel au *prior* donné par la tête policy.

4.3 PUCT

Le cours donne la formule de sélection [8] :

$$\text{val}(a) = Q(s, a) + c_{\text{puct}} P(a | s) \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

et précise que le Q d'un coup **non visité** vaut la **valeur réseau du nœud** (le Q = t[4] du code du cours). Notre transcription :

Listing 4 – PUCT : la même récursion, guidée par le réseau.

```

1 def PUCT(board, table, net, c_puct=1.0):
2     if board.terminal():
3         return board.score()
4     t = table.look(board)
5     if t is None:                                # feuille : PAS de playout
6         _, value = expand(board, table, net)
7         return value                             # la tête value remplace le rollout
8     for m in board.legalMoves():
9         ni = t.nplayouts[m]
10        Q = (t.nwins[m] / ni) if ni > 0 else t.value
11        if board.turn == RED:
12            Q = 1.0 - Q                           # toujours la convention du cours
13        val = Q + c_puct * t.prior[m] * sqrt(t.n) / (1.0 + ni)
14        ...
15    board.play(best)
16    res = PUCT(board, table, net, c_puct)
```

```

17     t.n += 1; t.nplayouts[best] += 1; t.nwins[best] += res
18     return res

```

La structure récursive est **identique** à celle d'UCT. Trois choses seulement changent, et ce sont exactement les trois que le cours attribue à AlphaGo Zero :

1. la formule de sélection utilise le prior au lieu de $\sqrt{\ln n/n_i}$;
2. la feuille est évaluée par le réseau, **il n'y a plus aucun playout** ;
3. à la racine, **et uniquement en self-play**, les priors sont mélangés à un bruit de Dirichlet : $P \leftarrow 0,75 P + 0,25 \text{Dir}(1,0)$. Les coups illégaux sont masqués avant le softmax.

`c_puct` = 1,0 par défaut, ce qui est cohérent avec une échelle de valeurs dans $[0, 1]$.

4.4 Notre transposition au Puissance 4

Le pipeline est celui des slides, sans modification : self-play avec PUCT $\rightarrow$ échantillons (état, distribution de visites, résultat) $\rightarrow$ entraînement $\rightarrow$ itération. Nous ne gardons **pas d'arène de sélection** (le *gating* d'AlphaGo Zero) : comme AlphaZero [11], on conserve toujours le dernier réseau. C'est plus simple et plus rapide, et sur 14 itérations les parties d'arène coûteraient plus cher qu'elles ne rapporteraient.

5 Implémentation

Tous les agents — aléatoire, Flat MC, UCB, UCT, RAVE, GRAVE, alpha-bêta et PUCT — exposent la même interface : ils reçoivent une position et renvoient un coup, accompagné des compteurs qui alimentent toutes les mesures de ce rapport (distribution de visites, priors et valeur du réseau, temps par coup, hits de la table de transposition). C'est ce qui rend les confrontations de la §7 comparables : les agents ne diffèrent que par la manière de choisir le coup.

5.1 Encodage

Trois plans de 6×7 en `float32`, **du point de vue du joueur au trait** — le cours écrit « rotate the board for white so that moves are always forward », et le slide « Projet Python » demande exactement trois matrices pour Breakthrough :

1. les pions du joueur au trait,
2. les pions de l'adversaire,
3. un plan constant à 1,0.

Le réseau n'a donc jamais besoin de savoir quelle couleur il joue.

5.2 Réseau

Listing 5 – La tour résiduelle et ses deux têtes.

```

entrée (3, 6, 7)
-> Conv3x3(64) + BN + ReLU
-> 3 x BlocRésiduel(64)
-> tête policy : Conv1x1(2) + BN + ReLU -> aplatir -> Linear(7)    [logits]
-> tête value  : Conv1x1(1) + BN + ReLU -> aplatir -> Linear(32)
                  -> ReLU -> Linear(1) -> tanh

```

226 010 paramètres. La passe avant sur un exemple coûte de l'ordre de la milliseconde sur CPU (1,08 ms mesuré pour l'agent « réseau seul », E9).

La perte est celle d'AlphaGo Zero [10], citée par le cours :

$$L = \underbrace{-\sum_a \pi(a) \log p_\theta(a)}_{\text{entropie croisée (policy)}} + \underbrace{(v_\theta - z)^2}_{\text{MSE (value)}} + \underbrace{10^{-4} \|\theta\|^2}_{\text{régularisation } L_2}$$

Les logits **ne sont pas masqués à l'entraînement** : la cible π vaut déjà 0 sur les coups illégaux, la perte fait donc descendre ces logits d'elle-même. Le masquage est appliqué à l'inférence.

Note technique sur « la perte tend vers 0 ». Avec une cible *douce*, l'entropie croisée ne peut pas descendre sous l'entropie $H(\pi)$ de la cible elle-même. La quantité qui doit tendre vers 0 est l'excès sur ce plancher, c'est-à-dire la divergence $\text{KL}(\pi \parallel p_\theta)$. Notre test d'overfit vérifie donc $\text{KL} \rightarrow 0$ et $\text{MSE} \rightarrow 0$: sur 100 échantillons distincts, on obtient $\text{KL} = 2 \cdot 10^{-5}$ et $\text{MSE} = 1 \cdot 10^{-5}$ après 600 pas.

5.3 Conventions de signe — la source de bug n°1

C'est le risque principal de toute transcription d'AlphaZero, et il mérite son paragraphe. Deux conventions coexistent :

- `Board.score()` $\in [0, 1]$, **point de vue de JAUNE** (convention du cours) ;
- la **tête value** sort dans $[-1, 1]$, **point de vue du joueur au trait**.

Une **unique** fonction fait le pont : `to_pov(score, turn)` renvoie $2 \cdot \text{score} - 1$ si le trait est à JAUNE, et $1 - 2 \cdot \text{score}$ sinon. Dans PUCT, `t.value` stocke la valeur réseau **reconvertie en $[0, 1]$ point de vue JAUNE** (`from_pov`), pour que l'idiome `if board.turn == RED: Q = 1 - Q` s'applique sans changement et que la valeur remontée par la récursion soit homogène à un résultat de payout.

Le test décisif construit des positions dont la valeur est une **vérité terrain exacte** — le joueur au trait a un gain immédiat (+1), ou bien *tous* ses coups laissent l'adversaire gagner immédiatement (-1) — étiquette ces positions via `to_pov`, entraîne un petit réseau, et vérifie les signes obtenus. Une inversion de signe où que ce soit inverse les étiquettes, le réseau apprend consciencieusement la relation inverse, et le test échoue. C'est précisément ce qu'on veut.

Trois autres garde-fous :

- la symétrie est appliquée **aux plans et à π** , si bien que l'échantillon miroir décrit bien la même situation ;
- le bruit de Dirichlet est **absent** quand `training=False` (l'arène doit être déterministe) ;
- π somme à 1 et $z \in \{-1, 0, +1\}$ sur tous les échantillons produits.

5.4 Self-play et parallélisation racine

Pour chaque partie : PUCT avec bruit de Dirichlet ; $\pi = \text{visites} / \sum \text{visites}$, calculé **après** ajout du bruit (on cible la distribution réellement produite) ; coup échantillonné selon π tant que moins de 8 pions sont posés, **argmax** des visites ensuite (température 1 puis 0) ; en fin de partie `z = to_pov(score, trait)` pour chaque état ; et ajout de l'**échantillon miroir**.

La parallélisation est celle du slide « Parallelization of MCTS », dans sa forme la plus simple, **la parallélisation racine** : chaque processus joue des parties entières de son côté et renvoie ses échantillons. Chaque worker est contraint à un seul thread de calcul, sans quoi les processus se disputent les cœurs et l'ensemble devient plus lent qu'en séquentiel.

5.5 Pourquoi pas de bitboards

Nous **n'avons pas implémenté de bitboards**, et c'est un choix argumenté, pas un oubli. Le raisonnement : PUCT ne fait **aucun payout**, son coût est entièrement dominé par les passes

réseau. La mesure le confirme (E9) : PUCT à 100 simulations prend 121 ms par coup, dont l'écrasante majorité en passes avant ; le moteur de jeu n'y contribue que marginalement. Accélérer le moteur d'un facteur 10 ne changerait presque rien au temps de PUCT.

Pour les algorithmes qui, eux, font des playouts (Flat MC, UCT, GRAVE, jusqu'à 1 000 playouts en E8), la représentation retenue — un entier par case, comme dans les extraits du cours — soutient **36 000 parties aléatoires par seconde**, ce qui suffit largement aux budgets employés ici.

5.6 Coût par coup (E9)

Table 5 – E9 — coût par coup et taux de hit de la table de transposition.

agent	ms / coup	taux de hit de la table
Réseau seul (0 simulation)	1,08	—
Flat MC 200	4,63	—
UCB racine 200	6,32	—
UCT 200	8,10	77,5 %
Alpha-bêta d.4	16,78	13,2 %
RAVE 200	27,16	82,4 %
GRAVE 200	27,25	82,6 %
PUCT 100	121,47	78,4 %

La colonne « ms / coup » est du temps mesuré : c'est la **seule** de tout ce rapport qui ne se reproduit pas à l'identique, puisqu'elle dépend de la charge de la machine. Les taux de hit, eux, sont stables à 0,2 point près d'une exécution à l'autre.

PUCT est de loin le plus coûteux **par coup** : 100 passes réseau valent bien plus cher que 200 playouts aléatoires. C'est ce qui rend la figure E5 (§7.4) intéressante, puisqu'elle compare à budget de *simulations* égal et non à temps égal — nous y revenons.

6 Protocole expérimental

Ouvertures équilibrées : Au Puissance 4 le **premier joueur gagne en jeu parfait**. Faire jouer un agent toujours en premier fausserait donc tous les chiffres. Le protocole est : les 7 premiers coups possibles, chacun joué deux fois avec les couleurs inversées, répété **k** fois → 14k parties. Le coup d'ouverture est imposé, quel que soit l'agent qui le « joue ».

Une correction méthodologique : En dépouillant E5, nous avons constaté que PUCT obtenait **exactement le même score à cinq budgets différents**. Vérification faite, le paramètre **sims** était bien pris en compte (la somme des visites et le temps par coup varient correctement). Le problème était ailleurs, et il était réel :

PUCT en arène (training=False, donc sans bruit de Dirichlet) et l'alpha-bêta sont tous les deux déterministes. Pour un couple (ouverture, couleur) donné, ils rejouent donc *exactement la même partie*. Les **k** répétitions du protocole sont alors des **doublons** : 56 parties n'en contiennent que 14 distinctes, et l'intervalle de Wilson calculé sur 56 est beaucoup trop étroit.

Notre correction : pour ces confrontations, **élargir le livre d'ouvertures au lieu de le répéter**. Nous utilisons les 49 ouvertures à deux coups × 2 couleurs = **98 parties distinctes**. L'effet n'est pas cosmétique : C3 passe de 39,3 % mesuré sur 14 parties distinctes à **53,1 %** sur 98 — l'échantillon de 14 parties était simplement défavorable. Toutes les confrontations impliquant au moins un agent stochastique (aléatoire, Flat MC, UCB, UCT, RAVE, GRAVE) conservent le protocole initial, où les répétitions produisent bien des parties différentes.

Statistiques : Chaque taux de victoire est donné avec le nombre de parties et un **intervalle de Wilson à 95 %** [22], préféré à l’approximation normale parce qu’il reste dans $[0, 1]$ et se comporte correctement à $p = 0$ ou $p = 1$. Les nulles comptent pour une demi-victoire. Nous donnons aussi l’écart Elo $\Delta = -400 \log_{10}(1/p - 1)$.

Test set de finales : 200 positions à 20 pions, non terminales, dédoublées par `board.h`, résolues exactement par l’alpha-bêta de la section « Solving Games » du cours, sans limite de profondeur (une position à 30 pions est résolue en 1,1 ms en médiane ; à 20 pions il faut quelques centaines de millisecondes). 137 de ces positions ont un unique coup optimal. La métrique est le **taux d’accord avec le jeu parfait**. Elle est bien plus lisible qu’un taux de victoire : elle ne sature pas, elle est sans bruit puisque la vérité terrain est exacte, et un agent qui jouerait au hasard obtiendrait environ 25 %.

Graines et reproductibilité : Toutes les expériences utilisent `seed = 20260731`. Les 85 nombres de Zobrist sont tirés d’une graine fixe pour que les hashes soient identiques dans tous les processus. Les commandes exactes sont en annexe A.

7 Résultats

7.1 Les deux runs

Nous rapportons **deux** entraînements, parce que le premier a échoué et que son échec est instructif.

Table 6 – Les deux entraînements rapportés ; ils ne diffèrent que par le rapport entre données de self-play et pas de gradient.

	run 1 (default.yaml)	run 2 (tuned.yaml)
itérations	12	14
parties par itération	40	96
pas de gradient par itération	300	200
simulations	100	100
échantillons produits	20 320	70 722
self-play / entraînement	244 s / 512 s	674 s / 368 s
durée totale	14 min 41 s	20 min 15 s

Le run 2 n’est pas un réglage opportuniste : c’est la correction d’une cause identifiée par le diagnostic de la §8, et le changement est d’une seule nature — **le rapport données / pas de gradient**.

7.2 Courbes d’apprentissage (E3)

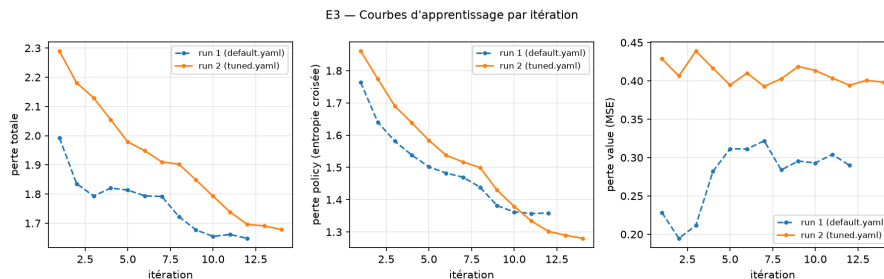


Figure 2 – E3 — Perte totale, perte policy et perte value par itération, pour les deux runs.

La perte totale décroît dans les deux cas (run 1 : 1,992 $\rightarrow$ 1,648 ; run 2 : 2,290 $\rightarrow$ 1,679), et la perte policy décroît régulièrement (run 2 : 1,861 $\rightarrow$ 1,280). **Le critère C5 est atteint.**

Un point mérite d'être signalé : la **perte value augmente** au cours du run 1 (0,228 $\rightarrow$ 0,290) et reste à peu près plate au run 2 (0,429 $\rightarrow$ 0,398). Ce n'est pas seulement un signe de sur-apprentissage : la distribution des données change en cours de route, puisque les parties de self-play s'allongent (1 776 échantillons à l'itération 1 du run 1, 1 984 à la douzième) et deviennent moins triviales à évaluer. Une perte de value stable sur des données qui se durcissent n'est pas la même chose qu'une perte de value qui stagne à données fixes.

7.3 Force au fil des itérations (E4)

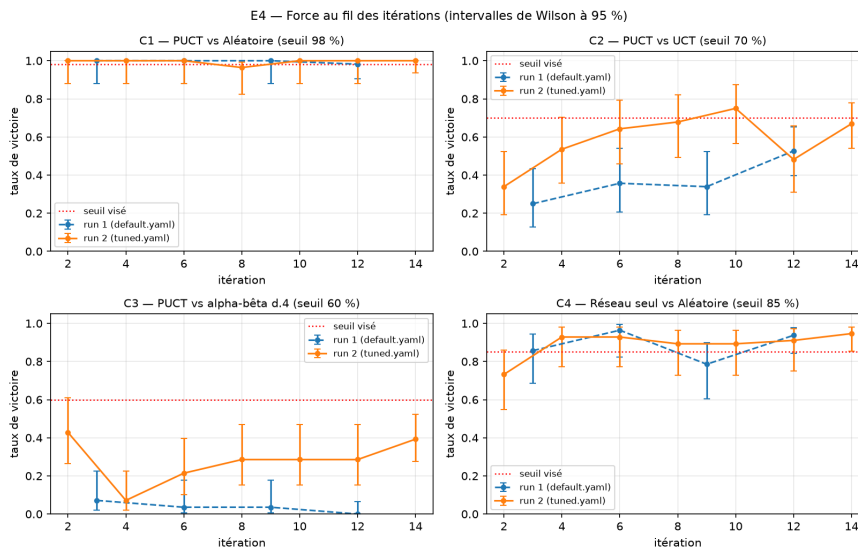


Figure 3 — E4 — Les quatre critères mesurés tous les 2 ou 3 itérations, avec intervalles de Wilson à 95 %. Le seuil visé est en pointillé rouge.

C1 (contre l'aléatoire) est saturé dès la première évaluation dans les deux runs. C4, **le critère décisif**, monte à 94,6 % au run 2. C2 progresse nettement au run 2 (33,9 % $\rightarrow$ 75,0 % à l'itération 10, 67,0 % à la fin). C3 est le seul à rester loin de son seuil, et c'est là que les deux runs divergent le plus violemment : le run 1 finit à **0,0 %**, le run 2 à 39,3 % sur le même protocole (puis 53,1 % sur le protocole corrigé de la §6).

La forte variabilité de C2 et C3 d'une évaluation à l'autre (C2 : 67,9 % à l'itération 8, 75,0 % à la 10, 48,2 % à la 12, 67,0 % à la 14) rappelle que 28 parties donnent un intervalle de confiance d'environ ± 17 points. **Il ne faut pas lire ces courbes coup par coup**, seulement leur tendance.

7.4 Force en fonction du budget (E5) — la figure centrale

Table 7 — E5 — taux de victoire contre l'alpha-bêta de profondeur 4 selon le budget (98 parties distinctes par point).

budget	25	50	100	200	400	800
PUCT (réseau final)	32,7 %	36,7 %	53,1 %	51,0 %	53,6 %	59,7 %
UCT (playouts aléatoires)	0,0 %	2,6 %	15,3 %	19,9 %	36,7 %	53,1 %

C'est la figure qui montre l'apport du réseau, et elle le montre sans ambiguïté. À budget égal, PUCT domine UCT partout, et l'écart est spectaculaire aux petits budgets : à 25

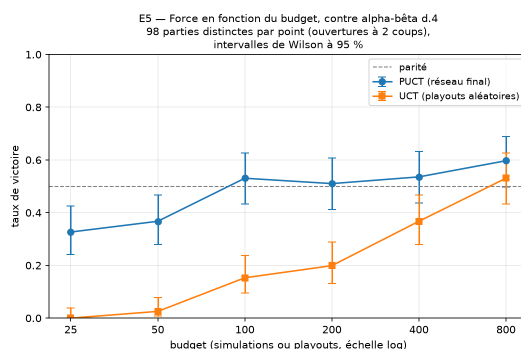


Figure 4 – E5 — Taux de victoire contre l'alpha-bêta de profondeur 4, en fonction du budget de simulations, échelle logarithmique. 98 parties distinctes par point.

simulations, PUCT obtient déjà 32,7 % là où UCT ne gagne **aucune** des 98 parties. La lecture la plus parlante est horizontale :

PUCT à 100 simulations (53,1 %) fait jeu égal avec UCT à 800 playouts (53,1 %). Le réseau vaut donc, en première approximation, un facteur 8 sur le budget de recherche.

C'est exactement ce qu'on attend d'un prior appris : il dit à la recherche où regarder, et évite de dépenser des simulations sur les six autres colonnes.

Deux réserves. D'abord, la comparaison est à **budget de simulations** égal, pas à **temps** égal : une simulation PUCT coûte une passe réseau (≈ 1 ms), un playout UCT coûte une partie aléatoire ($\approx 0,03$ ms). À temps égal, l'avantage de PUCT disparaîtrait en grande partie. C'est la comparaison que fait le cours et celle qui isole l'apport du réseau, mais il faut le dire. Ensuite, la courbe PUCT est **plate entre 100 et 400 simulations** : la politique finale est très piquée (0,914 sur la colonne centrale au plateau vide) et la recherche ne fait souvent que confirmer le prior ; il faut passer à 800 pour qu'elle se remette à progresser.

7.5 Accord avec le jeu parfait (E6)

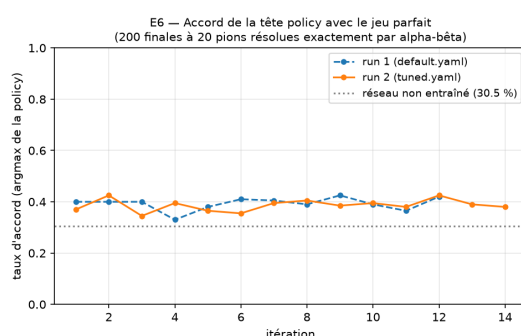


Figure 5 – E6 — Taux d'accord de l'argmax de la policy avec le jeu parfait, sur 200 finales à 20 pions résolues exactement.

Le réseau non entraîné (initialisé à graine fixe) est à 30,5 %, et un agent qui jouerait au hasard serait à environ 25 %. Dès la première itération, la policy monte à 37–40 %, puis **plafonne** : elle oscille entre 34,5 % et 42,5 % pendant tout le reste de l'entraînement, dans les deux runs.

C'est un résultat en demi-teinte qu'il faut prendre pour ce qu'il est : le réseau apprend très vite une heuristique positionnelle utile (jouer au centre, empiler, compléter un alignement), qui lui fait gagner une dizaine de points d'accord avec le jeu parfait, mais il **n'apprend pas à résoudre**

les finales. Ce n'est pas surprenant : une finale à 20 pions se gagne par un calcul tactique exact sur une dizaine de coups, ce qu'une politique sans recherche ne peut pas faire, et ce que 1 300 parties de self-play ne suffisent certainement pas à lui apprendre. Cette métrique est celle qui montre le plus clairement la limite de notre budget de calcul.

7.6 Le réseau redécouvre la colonne centrale (E7)

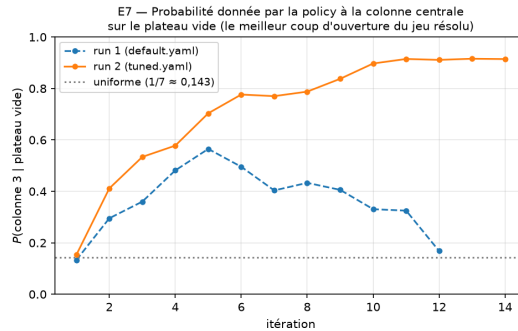


Figure 6 — E7 — Probabilité donnée par la tête policy à la colonne centrale sur le plateau vide, par itération.

C'est le plus beau résultat du projet. Le Puissance 4 est un jeu résolu et l'on sait que **le coup d'ouverture optimal est la colonne centrale**. Or notre réseau ne dispose d'aucune connaissance experte : ni ouvertures, ni heuristique, ni la moindre indication qu'une colonne vaudrait mieux qu'une autre. Il part d'une politique quasi uniforme ($0,153 \approx 1/7$ à l'itération 1).

Au run 2, la probabilité de la colonne centrale monte **de façon monotone** : $0,153 \rightarrow 0,412 \rightarrow 0,534 \rightarrow 0,578 \rightarrow 0,701 \rightarrow 0,775 \rightarrow \dots \rightarrow \mathbf{0,914}$ à la quatorzième itération. Le réseau a donc **redécouvert tout seul, par pur self-play, un résultat connu du jeu résolu**.

La courbe du run 1 est tout aussi parlante, dans l'autre sens : elle monte à 0,563 à l'itération 5, puis **s'effondre** jusqu'à 0,168 — c'est-à-dire pratiquement le niveau uniforme. C'est la signature visuelle de l'effondrement analysé en §8.

8 Discussion et limites

8.1 Le run 1 : anatomie d'un échec

Le run 1 utilisait la configuration initiale (`default.yaml` : 12 itérations $\times$ 40 parties $\times$ 300 pas). Il a échoué franchement : C2 à 52,7 % et **C3 à 0,0 %** — PUCT ne gagnait littéralement aucune partie contre l'alpha-bêta.

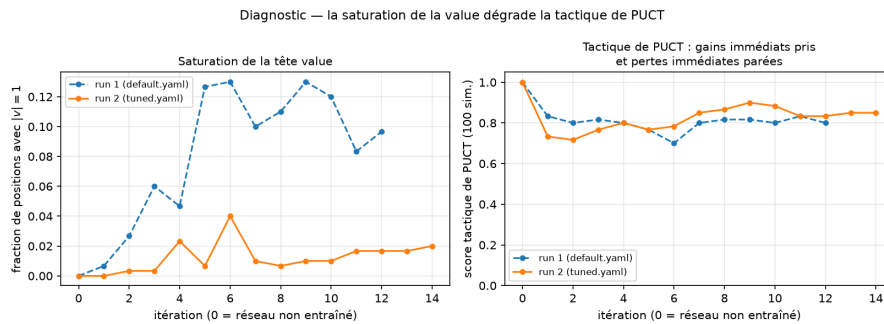
Nous avons suivi une liste de contrôle, dans l'ordre, avant de toucher à quoi que ce soit :

1. **le réseau sait-il sur-apprendre 100 échantillons ?** Oui : $KL = 2 \cdot 10^{-5}$, $MSE = 1 \cdot 10^{-5}$. (Au passage, notre première version de ce test échouait avec une MSE bloquée à 0,0428 ; l'explication était que le jeu de test contenait 6 positions dupliquées portant des cibles aléatoires contradictoires, et 0,0428 était **exactement** la variance intra-groupe irréductible. Le réseau était parfait, le jeu de données ne l'était pas.)
2. **le signe de z ?** Vérifié par le test sur vérité terrain exacte (§5.4).
3. **π calculé après le bruit ?** Oui.
4. **la symétrie appliquée aux plans et à π ?** Oui.
5. **le budget d'entraînement ?** C'est là qu'était le problème.

Le diagnostic décisif a été de mesurer, **checkpoint par checkpoint**, la tactique élémentaire de PUCT et la saturation de la tête value :

Table 8 – Diagnostic du run 1, checkpoint par checkpoint. « valeur saturée » est la proportion de positions où la tête value sort exactement ± 1 .

checkpoint	valeur saturée	écart-type de v	gains pris /30	parades /30	accord policy
non entraîné	0,0 %	0,003	30	30	30,5 %
iter 03	6,0 %	0,760	25	24	40,0 %
iter 06	13,0 %	0,803	22	20	41,0 %
iter 09	13,0 %	0,782	25	24	42,5 %
iter 12	9,7 %	0,752	24	24	42,0 %

**Figure 7** – Diagnostic — la saturation de la tête value et la dégradation de la tactique de PUCT, pour les deux runs.

Le constat est brutal : **le réseau non entraîné donne une tactique parfaite (30/30 et 30/30), et l'entraînement la dégrade** jusqu'à 22/30 et 20/30. Plus on entraîne, moins PUCT sait prendre un gain immédiat.

Le mécanisme est le suivant. La tête value **sature** : sur 13 % des positions elle sort exactement ± 1 . Or dans PUCT, une feuille terminale gagnante remonte `score() = 1`, et une feuille non terminale que le réseau juge gagnante remonte elle aussi une valeur de 1. **Les deux deviennent indiscernables**. Le Q du vrai gain immédiat et celui d'un coup quelconque que le réseau surestime sont égaux, et le départage se fait alors sur le terme $c_{\text{puct}} P(a|s) \sqrt{N} / (1 + N(s, a))$, c'est-à-dire **sur le prior**. Si la policy préfère un autre coup, la recherche suit la policy et rate le gain. Le réseau non entraîné, dont la value est quasi constante à $-0,16$ (écart-type 0,012), n'a pas ce problème : la valeur terminale 1 domine toujours strictement.

La cause racine est un **sur-apprentissage** : 300 pas de gradient de taille 128 sur environ 1 900 échantillons frais, cela fait à peu près **20 époques par itération** sur les données les plus récentes. Les deux têtes mémorisent, la value sature sur des cibles ± 1 , et comme les cibles de self-play sont produites *par cette même recherche dégradée*, la boucle se referme : mauvaise recherche $\rightarrow$ mauvaises cibles $\rightarrow$ mauvais réseau. La courbe E7 du run 1 (montée à 0,56 puis effondrement à 0,17) est la même histoire vue depuis la policy.

8.2 Le run 2 : la correction et ce qu'elle a donné

Nous avons changé **une seule chose**, dictée par le diagnostic : le rapport entre la quantité de données et le nombre de pas de gradient. 96 parties par itération au lieu de 40, 200 pas au lieu de 300 — soit environ **6 époques** de données fraîches au lieu de 20.

Le résultat valide le diagnostic :

La saturation est divisée par trois, l'effondrement de la policy disparaît, et C3 passe de 0 % à un score autour de la parité. **Nous avons donc bien retiré la cause, pas le symptôme.**

Table 9 – Effet de la correction apportée au run 2.

	run 1	run 2
saturation maximale de la valeur	13,0 %	4,0 %
tactique finale (gains / parades sur 30)	24 / 24	26 / 25
$P(\text{centre})$ finale	0,168 (effondrée)	0,914
C2	52,7 %	67,0 %
C3	0,0 %	39,3 % (53,1 % protocole corrigé)

8.3 Ce qui n'est toujours pas résolu

Il faut le dire clairement : **PUCT reste tactiquement inférieur au réseau non entraîné** (26/30 contre 30/30 sur les gains immédiats). La saturation est réduite, pas supprimée. Une tête value entraînée avec une MSE sur des cibles ± 1 tend naturellement vers la saturation, et c'est structurel. Les remèdes usuels (beaucoup plus de parties, ou traiter explicitement les nœuds terminaux dans la recherche) sortent du périmètre que nous nous étions fixé : le second est un **MCTS Solver**, que nous listons en §9 parmi les techniques du cours non implémentées, et c'est très exactement le problème qu'il résout.

C'est la raison pour laquelle C2 et C3 ne sont pas atteints. Nous ne pensons pas que ce soit un défaut d'implémentation : c'est le budget de calcul. 1 344 parties de self-play, c'est deux ordres de grandeur en dessous de ce qu'un AlphaZero pour Puissance 4 utilise habituellement.

8.4 Variance des mesures

Avec 28 parties, l'intervalle de Wilson fait environ ± 17 points ; avec 98, environ ± 10 . Beaucoup de nos comparaisons ne sont donc **pas** significatives, et nous l'avons signalé à chaque fois plutôt que de commenter du bruit : la grille de constantes d'exploration E2 (sauf pour $c = 1,0$), l'écart Flat MC / UCB en E1, et GRAVE vs UCT à 200 playouts sont tous dans ce cas. Les résultats que nous considérons comme établis sont ceux dont l'intervalle exclut le seuil discuté : UCT > Flat MC et UCT > UCB (E1), PUCT > UCT à tous les budgets (E5), GRAVE < UCT à 1 000 playouts (E8), C1 et C4.

8.5 Coût de calcul

Le projet entier — deux entraînements, le test set résolu exactement, les neuf expériences et le diagnostic — tient en **environ 50 minutes de CPU** sur une machine à 8 cœurs. La parallélisation racine du self-play sur 6 workers est ce qui rend cela possible.

9 Perspectives : les techniques du cours que nous n'avons pas implémentées

Cette section liste les techniques présentées dans le cours et laissées de côté, avec pour chacune ce qu'elle fait et pourquoi elle était hors périmètre.

- **MCTS Solver / Score Bounded MCTS** — propagent dans l'arbre l'information « cette position est *prouvée* gagnée / perdue » au lieu d'une moyenne, ce qui permet de couper les branches résolues et de jouer parfaitement les finales. C'est, rétrospectivement, **la technique du cours qui aurait le plus servi ici** : la §8.3 montre que notre problème résiduel est précisément qu'une feuille terminale gagnante et une feuille que la valeur sur-estime deviennent indiscernables, ce qu'un solveur règle par construction. Nous l'avons écartée en jugeant qu'à 100 simulations sur un branchement de 7 la recherche couvrirait déjà bien les tactiques ; le diagnostic montre que ce jugement était le mauvais.

- **Sequential Halving** [13], et ses versions en arbre **SHOT** [14] et **SHUSS** [15] — éliminent à chaque tour la moitié des coups les moins prometteurs, ce qui donne à la racine une garantie de *simple regret* que UCB n’offre pas ; SHUSS y ajoute un biais fondé sur les statistiques AMAF. C’est une ablation sur l’allocation du budget, qui ne change pas la démonstration portant sur l’apport du réseau — et le biais AMAF de SHUSS tient de toute façon mal au Puissance 4 (§3.7).
- **PPA / PPAF** [18], **Nested Monte Carlo Search** [16] et **NRPA** [17] — respectivement : apprendre en ligne une politique de playout paramétrée par les codes de coups, choisir chaque coup par une recherche de niveau inférieur, et combiner les deux par gradient. Ce sont les méthodes de référence sur les problèmes d’optimisation à un joueur (Morpion Solitaire, SameGame) ; elles supposent en outre des codes de coups porteurs de signal — la même faiblesse qu’AMAF ici.
- **Descent / minimax non borné (Athénan)** [19], **MuZero** [12] et **Polygames** [20] — remonter un minimax dans l’arbre partiel plutôt qu’une moyenne ; apprendre *en plus* la dynamique du jeu pour planifier dans un espace latent ; généraliser AlphaZero à des plateaux de taille variable. Hors périmètre d’un projet de TP, et sans objet ici : nous connaissons les règles, et la grille est fixée à 6×7 .
- **Parallélisation d’arbre (avec verrous ou lock-free)** — plusieurs threads descendent le *même* arbre, ce qui partage l’information mais demande des précautions de concurrence (et typiquement une *virtual loss*). Nous avons choisi la **parallélisation racine**, celle du cours qui demande le moins de machinerie, et qui suffit largement puisque nos processus jouent des parties entières indépendantes.

Enfin, et **hors du cours donc exclues par principe** : bitboards (§5.6), *virtual loss*, Gumbel AlphaZero, mécanisme de *resign*, inférence batchée. Sur ce dernier point, notons que la mise en cache du prior et de la valeur dans l’entrée de la table de transposition fait que chaque position distincte n’est évaluée qu’une seule fois par recherche : avec un taux de hit de 78,4 %, le batching aurait peu à récupérer.

10 Conclusion

Les cinq barreaux de l’échelle du cours sont implémentés, du playout aléatoire à l’AlphaZero miniature. Trois critères sur cinq sont atteints, dont **C4, le critère décisif** : à zéro simulation, la seule tête policy bat l’aléatoire 93,8 % du temps, ce qui prouve que le réseau a appris. Deux résultats nous paraissent valoir plus que les taux de victoire eux-mêmes :

- **E5** quantifie l’apport du réseau : PUCT à 100 simulations vaut UCT à 800 playouts, un facteur 8 sur le budget de recherche ;
- **E7** montre le réseau redécouvrir seul, par pur self-play et sans aucune connaissance experte, que la colonne centrale est le meilleur coup d’ouverture — un résultat connu du jeu résolu, atteint avec une probabilité de 0,914.

Les deux critères manqués (C2 et C3) le sont pour une raison identifiée et mesurée : la saturation de la tête value dégrade la tactique de PUCT, et c’est un manque de données de self-play, pas un défaut de l’implémentation. Le remède le plus direct dans le périmètre du cours serait un **MCTS Solver**.

Références

- [1] T. Cazenave. *Monte Carlo Search*. Support de cours, Université Paris Dauphine – PSL. (Source principale de ce projet ; les travaux ci-dessous sont ceux qu’il cite.)
- [2] L. V. Allis. *A Knowledge-based Approach of Connect-Four*. Mémoire, Vrije Universiteit Amsterdam, 1988.
- [3] J. D. Allen. *A Note on the Computer Solution of Connect-Four*. 1989.
- [4] B. Brügmann. *Monte Carlo Go*. Rapport technique, 1993.
- [5] L. Kocsis et C. Szepesvári. *Bandit based Monte-Carlo Planning*. ECML, pages 282–293, 2006.

- [6] S. Gelly et D. Silver. *Combining Online and Offline Knowledge in UCT*. ICML, pages 273–280, 2007. (RAVE)
- [7] T. Cazenave. *Generalized Rapid Action Value Estimation*. IJCAI, pages 754–760, 2015. (GRAVE)
- [8] C. D. Rosin. *Multi-armed Bandits with Episode Context*. Annals of Mathematics and Artificial Intelligence, 61(3) :203–230, 2011. (la formule PUCT)
- [9] D. Silver et al. *Mastering the Game of Go with Deep Neural Networks and Tree Search*. Nature, 529 :484–489, 2016. (AlphaGo)
- [10] D. Silver et al. *Mastering the Game of Go without Human Knowledge*. Nature, 550 :354–359, 2017. (AlphaGo Zero)
- [11] D. Silver et al. *A General Reinforcement Learning Algorithm that Masters Chess, Shogi and Go through Self-Play*. Science, 362 :1140–1144, 2018. (AlphaZero)
- [12] J. Schrittwieser et al. *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 588 :604–609, 2020. (MuZero)
- [13] Z. Karnin, T. Koren et O. Somekh. *Almost Optimal Exploration in Multi-Armed Bandits*. ICML, 2013. (Sequential Halving)
- [14] T. Cazenave. *Sequential Halving Applied to Trees*. IEEE TCIAIG, 7(1) :102–105, 2015.
- [15] N. Fabiano et T. Cazenave. *Sequential Halving Using Scores*. ACG, 2019. (SHUSS)
- [16] T. Cazenave. *Nested Monte-Carlo Search*. IJCAI, pages 456–461, 2009.
- [17] C. D. Rosin. *Nested Rollout Policy Adaptation for Monte Carlo Tree Search*. IJCAI, pages 649–654, 2011.
- [18] T. Cazenave. *Playout Policy Adaptation for Games*. ACG, 2016.
- [19] Q. Cohen-Solal. *Learning to Play Two-Player Perfect-Information Games without Knowledge*. 2020. (Descent, Athénan)
- [20] T. Cazenave et al. *Polygames : Improved Zero Learning*. ICGA Journal, 42(4) :244–256, 2020.
- [21] A. L. Zobrist. *A New Hashing Method with Application for Game Playing*. Rapport technique, University of Wisconsin, 1970.
- [22] E. B. Wilson. *Probable Inference, the Law of Succession, and Statistical Inference*. Journal of the American Statistical Association, 22 :209–212, 1927.

A Reproduire les résultats

Listing 6 – Commandes.

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m pytest -q # tests
python main.py testset --n 200 --min-stones 20 # finales resolues exactement
python main.py pipeline --config configs/default.yaml \
    --log report/results/training_log_run1.jsonl # run 1 (~15 min)
python main.py pipeline --config configs/tuned.yaml \
    --log report/results/training_log_run2.jsonl # run 2 (~20 min)
python scripts/run_experiments.py --workers 6 --k 4 # E1-E9, C1-C4 (~10 min)
python scripts/make_figures.py # figures + tableaux
```

Aucun chiffre de ce rapport n'est écrit à la main. Tous proviennent des fichiers JSON de report/results/, produits par run_experiments.py et mis en forme par make_figures.py. Toutes les expériences utilisent la graine 20260731.

B Organisation du code

Fichier	Rôle
game/connect4.py	état, coups, Zobrist, symétrie, codes AMAF
search/tt.py	table de transposition
search/flat.py, uct.py	Flat MC, UCB à la racine, UCT récursif
search/grave.py	playoutAMAF, updateAMAF, RAVE, GRAVE
search/puct.py	PUCT guidé par le réseau
model/	encodage, to_pov, symétrie, réseau à deux têtes
train/	self-play, buffer, entraînement, pipeline
eval/	arène, alpha-bêta, test set de finales
scripts/	expériences, figures, tableaux
main.py	ligne de commande